Mark	

Team name	*A10*
Homework number	*HOMEWORK 08*
Due date	13/11/2026

Contribution	NO	Partial	Full
Carbone Alessandro			*X*
Casareale Alberto			*X*
Colombi Federico			*X*
Lorusso Luca			*X*
Martorano Gabriele			*X*
Notes: Everyone contributed to understanding the logic and implementing the code			

Project name	Temperature sensor over I2C (full resolution, interrupt)		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To achieve the objective of reading the full 11-bit resolution temperature from the LM75B sensor over I2C and sending it to a remote terminal every 1 second, we first consulted the "Hands-on Lab Schematics.pdf". We verified that the sensor's I2C lines are connected to pins PB8 (I2C1_SCL) and PB9 (I2C1_SDA). In STM32CubeMX, we configured I2C1 in Standard Mode (100 kbps). To maximize efficiency and prevent CPU blocking, we configured the Direct Memory Access (DMA) for both I2C data reception (I2C1_RX) and USART data transmission (USART2_TX). We enabled all corresponding interrupts in the NVIC: the I2C1 Event and Error interrupts, the USART2 global interrupt, and the global interrupts for the specific DMA streams. Finally, to handle the precise 1-second timing, Timer 2 (TIM2) was configured with its global interrupt enabled.

Part 2: Code Generation and Verification

After generating the project code, we verified the TIM2 initialization parameters in the main.c file. To establish a precise 1-second interval using an 84 MHz system clock, the prescaler (PSC) was set to 8400-1 and the counter period (ARR) to 10000-1. Before compiling, to properly format floating-point numbers with printf, we explicitly passed the -u _printf_float linker flag by adding target_link_options (${CMAKE_PROJECT_NAME} PRIVATE -u _printf_float) directly into our CMakeLists.txt file. In the main() function, immediately before the infinite loop, the system is armed by starting Timer 2 in interrupt mode using HAL_TIM_Base_Start_IT(&htim2). The while(1) loop is left completely empty to save CPU resources.

Part 3: Functional Logic Implementation

The entire functional logic of the system is strictly event-driven and handled in the background through a chain of hardware interrupts and DMA transfers. Every second, Timer 2 automatically fires the HAL_TIM_PeriodElapsedCallback. Inside this routine, the microcontroller initiates an asynchronous DMA reading of 2 bytes from the LM75B's temperature register (address 0x00 at device address 0x90) by calling HAL_I2C_Mem_Read_DMA.

As soon as the DMA controller finishes transferring the two bytes into the memory buffer, it automatically triggers the HAL_I2C_MemRxCpltCallback. Within this callback, the raw data is processed: the MSByte and LSByte are merged into a 16-bit signed integer (int16_t). Since the LM75B provides an 11-bit left-justified value in two's complement format, the merged 16-bit variable is shifted right by 5 positions (raw_temp >> 5). Because a signed integer is used, the C compiler performs an arithmetic right shift, seamlessly preserving the sign bit for negative temperatures.

The aligned raw value is then multiplied by the sensor's resolution (0.125°C per bit) to obtain the actual temperature as a floating-point number. This value is securely formatted into a string with 3 decimal places using sprintf. Finally, the string is sent to the terminal via the non-blocking HAL_UART_Transmit_DMA function, completely offloading the transmission process to the hardware and completing the 1-second cycle efficiently without ever stalling the main program execution.

Professor comments: